Roy and Coin Boxes

Visual Algorithm Breakdown

Optimal Prefix Sum & Frequency Array Approach • O(N) Time Complexity

💡 **Want to save this as a PDF?**
Press `Ctrl/Cmd + P` and select "Save as PDF". Margins are already optimized.

Save as PDF

🎯 The Problem (Kya Karna Hai?)

There are `N` coin boxes (initially empty).

For `M` days, an operation provides a range `[L, R]`. You drop 1 coin into every box from index L to R.

The Query:
You will be given `Q` queries. Each query gives a number `X`. You must answer:
"How many boxes have at least X coins?"

1 Range Addition (Difference Logic)

Doing a loop from L to R for every query is **O(N*M) TLE** . Instead, we use two arrays to mark the start and end of ranges.

L[ ] Array

Increments at the START of a range.

`L[start_idx]++`

R[ ] Array

Increments at the END of a range.

`R[end_idx]++`

Example Update: Range [2, 4]

Magic Formula: `F[i] = F[i-1] + L[i] - R[i-1]`

Current Coins = Previous Coins + New Ranges Starting Here - Ranges that ended just before this.

2 Frequency Array (Kitne coin wale kitne box?)

Once we have the total coins in each box `F[i]`, we need to answer `>= X` queries quickly. First, we count exactly how many boxes have exactly `C` coins.

Final Coins array F[ ]

Box	1	2	3	4	5
Coins	2	3	2	1	1

→

Frequency array freq[ ]

Coins (Idx)	1	2	3
Count	2	2	1

(e.g., 2 boxes have exactly 1 coin)

```
for(int i=1; i<=size; i++) { freq[ f[i] ]++; }
```

3 Suffix Sum (Ulta Prefix Sum)

A query asks for **"at least X"** (>= X) coins. So for Query(2), the answer is: `(boxes with 2 coins) + (boxes with 3 coins) + (boxes with 4 coins)...`

Reverse Addition Visualization

```
suffix_sum[i] = suffix_sum[i+1] + freq[i]
```

🎉 Final Step: For any query X, simply print `suffix_sum[X]` in O(1) time!